

We present a workspace architecture in which an agent session, its file tree, and its rendered artifacts share a single surface. The design keeps a strict separation between a rendering-only frontend and a sidecar that owns all business logic, and we show that the resulting system sustains trees with hundreds of thousands of nodes without measurable frame cost.

Tool-using agents are usually constrained by the surface they can observe. A flat file list, a paginated tree, or a transcript without inline artifacts all force the model into a lossy summary of its own workspace. We argue that the file tree is the primary context surface for research work, and that a session which can address any node of that tree, at any depth, without loading it into a dialog, is materially more capable.

Our contribution is architectural rather than algorithmic. The renderer stores a normalized map and derives rows on demand; the sidecar owns the session and emits a newline-delimited protocol; the boundary between them is a single total function that classifies frames.

Earlier editors modelled the workspace as a widget. Linear-time tree projections, virtual scrolling, and incremental stores are all well understood individually; the novelty here is refusing to persist any of them in the view layer.

Scaling Agent Sessions to Infinite Workspaces

Research Group OMP

omp-labs: preprint, October 2026

1 Introduction

2 Related Work

Abstract

3.1 Normalized store

The workspace is stored as a flat map from path to node, so that every mutation touches $O(1)$ entries regardless of tree depth:

3 Method

$S : \text{Path} \rightarrow \text{Node}$, $\text{Node} = (\text{kind}, \text{name}, \text{children}, \text{meta})$

Rendering is derived, never persisted. A projection walks only expanded directories and yields a flat row list, which is then windowed:

Raw frames arrive as JSON text. The renderer applies a total function from frames to UI

3.2 Boundary translation

events, any during the translation cannot classify is surfaced as an error rather than dropped, so protocol drift is loud instead of silent.

We generated synthetic workspaces of increasing size and measured the time to produce a stable frame set in an expand operation.

4 Results

Table 1: Expand latency is dominated by the first full projection, which is paid once per session and then

cached. Frame cost is flat because the windowed row set never exceeds the viewport, independent of workspace size.

Nodes	Tree build	First paint	Frame cost
1,000	0.9 ms	4.1 ms	0.2 ms
50,000	6.4 ms	5.0 ms	0.3 ms
500,000	71.0 ms	5.6 ms	0.4 ms

Splitting the surface has a cost: the renderer must be explicit about what it does not know. In practice this is a feature. Every capability the UI lacks is a capability the sidebar can add without a frontend release, and every assumption the UI makes is a single translation away from being wrong in exactly one place.

The remaining risk is drift between the protocol and its documentation. We mitigate it by treating unknown frames as errors rather than no-ops, and by pinning the frame enum in a single crate shared by both sides.

References that scales with the number of files, not the number of rendered rows, is straightforward once the tree is treated as derived state. The protocol that carries an agent session through that workspace is the part worth getting right.

[1] R. Steele. Newline-Delimited JSON. Self-published, 2024.

[2] Research Group Omp. Tool Use in Long-Horizon Sessions. arXiv, 2025.

[3] A. Vaswani et al. Attention Is All You Need. NeurIPS, 2017.